RAG Application Assignment Report

1. Overview

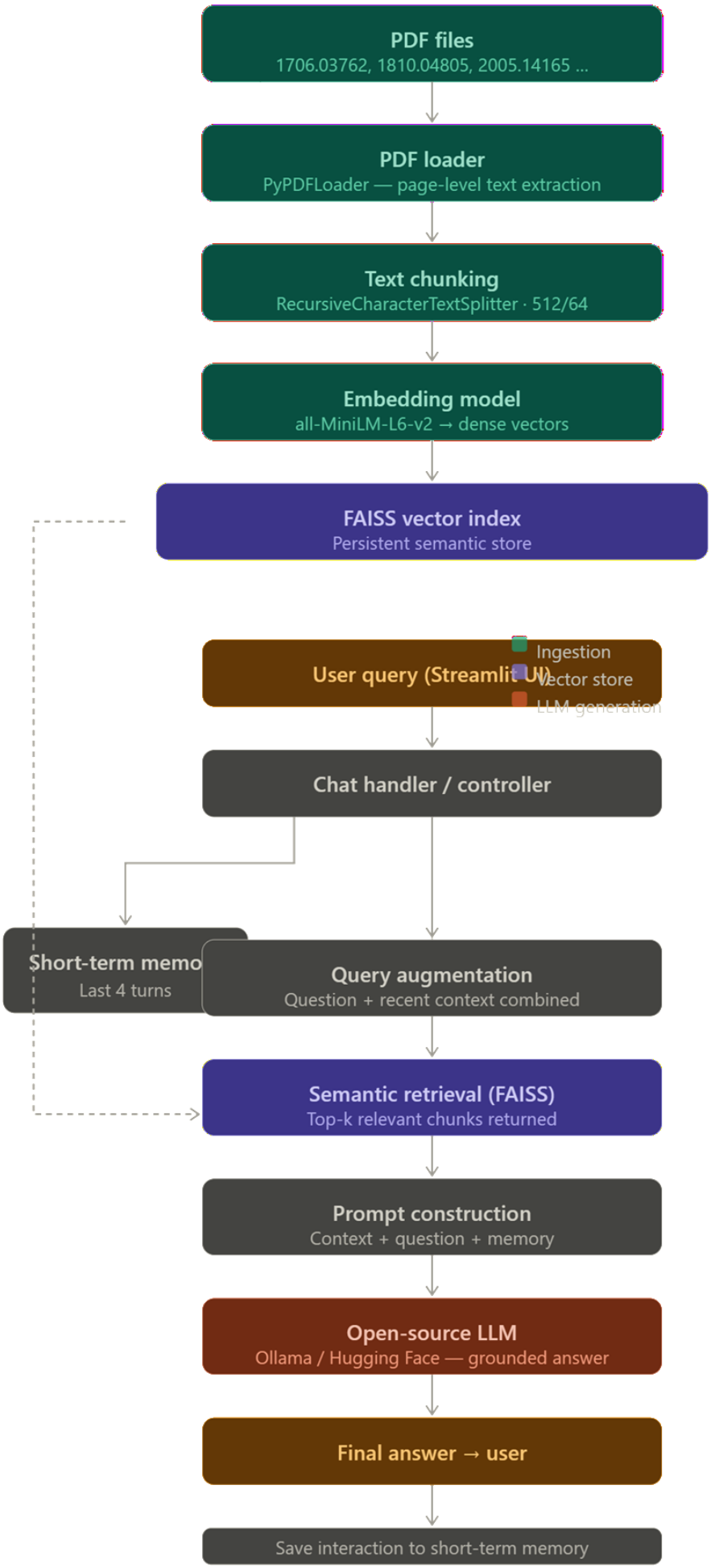
This project implements a local Retrieval-Augmented Generation (RAG) chatbot over five research papers provided in PDF format. The goal of the system is to ingest the PDFs, transform their contents into vector embeddings, store them in a FAISS vector database, retrieve semantically relevant chunks for a user query, and generate answers using an open-source language model. The chatbot maintains short-term conversational memory over the last four interactions.

The system is organized as a modular pipeline with separate components for configuration, document ingestion, embedding and indexing, retrieval, language model routing, conversation handling, memory retention, Streamlit UI, and evaluation.

At a high level, the application flow is:

1. Load PDF files from the docs/ directory.
2. Extract page text using a PDF loader.
3. Split extracted text into overlapping chunks.
4. Generate embeddings for the chunks.
5. Store embeddings in a FAISS vector index.
6. Accept a user query through the Streamlit interface.
7. Retrieve relevant chunks from FAISS.
8. Pass the retrieved context and question to an open-source LLM.
9. Return the generated answer and store the interaction in short-term memory.

Below is the flow diagram for the above mentioned architecture :-


PDF files
1706.03762, 1810.04805, 2005.14165 ...
PDF loader
PyPDFLoader — page-level text extraction
Text chunking
RecursiveCharacterTextSplitter · 512/64
Embedding model
all-MiniLM-L6-v2 → dense vectors
FAISS vector index
Persistent semantic store
Ingestion
Vector store
LLM generation
User query (Streamlit UI)
Chat handler / controller
Short-term memo
Last 4 turns
Query augmentation
Question + recent context combined
Semantic retrieval (FAISS)
Top-k relevant chunks returned
Prompt construction
Context + question + memory
Open-source LLM
Ollama / Hugging Face — grounded answer
Final answer → user
Save interaction to short-term memory

The main source files involved in this architecture are:

- rag/embedder.py
- rag/pipeline.py
- models/llm_router.py
- models/ollama_model.py
- models/hf_model.py
- conversation/chat_handler.py
- conversation/session_store.py
- ui/app.py

Configuration is centralized in the Settings class in config/settings.py, accessed via the get_settings() function.

2. Technical Implementation

2.1 PDF Ingestion and Data Extraction

The project uses the five required PDFs stored in the docs/ folder:

- 1706.03762.pdf
- 1810.04805.pdf
- 2005.14165.pdf
- 1907.11692.pdf
- 1910.10683.pdf

Document ingestion is implemented in rag/embedder.py. The core method is load_documents(), which scans the document directory and loads PDF files using PyPDFLoader. Each loaded page is stored as a LangChain Document, and metadata is enriched with the originating filename using the source_file field.

Relevant method:

- load_documents(docs_dir: Path = DOCS_DIR, arxiv_urls: Optional[List[str]] = None)

There is also a helper method download_pdf_from_url() that supports downloading the assignment PDFs directly from arXiv before indexing.

Relevant method:

- download_pdf_from_url(url: str, docs_dir: Path = DOCS_DIR)

This ingestion pipeline focuses on extracting textual content from the PDFs , and provide reliable page-based extraction suitable for downstream semantic retrieval.

2.2 Preprocessing and Chunking

After extraction, documents are segmented using RecursiveCharacterTextSplitter in the chunk_documents() method in rag/embedder.py.

Relevant method:

- chunk_documents(docs: List[Document])

The chunking configuration is:

- chunk_size = 512
- chunk_overlap = 64
- Separators: ["\n\n", "\n", ". ", " ", ""]

This design attempts to preserve sentence and paragraph co-relations while producing chunks small enough for efficient embedding and retrieval. The overlap helps reduce information loss at chunk boundaries.

2.3 Embedding Generation

Embeddings are generated using a Sentence Transformers model through LangChain's Hugging Face embedding wrapper. The model is initialized in the get_embedder() function in rag/embedder.py.

Relevant method:

- get_embedder() -> HuggingFaceEmbeddings

Configured embedding model:

- sentence-transformers/all-MiniLM-L6-v2

The embedder is cached with @lru_cache() so that it loads once per process. It is configured to run on CPU and normalize embeddings for more stable similarity search.

2.4 Vector Database Setup

The vector database is built using FAISS. The logic is implemented in build_index() and load_index() in rag/embedder.py.

Relevant methods:

- build_index(chunks: List[Document]) -> FAISS
-
- creates the FAISS index from chunk embeddings,
- persists it to the configured faiss_store directory,

- saves both index.faiss and index.pkl.

The current saved FAISS index exists in the repository and loads successfully. It contains 1397 vectors, which confirms that the five PDFs were ingested and indexed.

The indexing script is provided in:

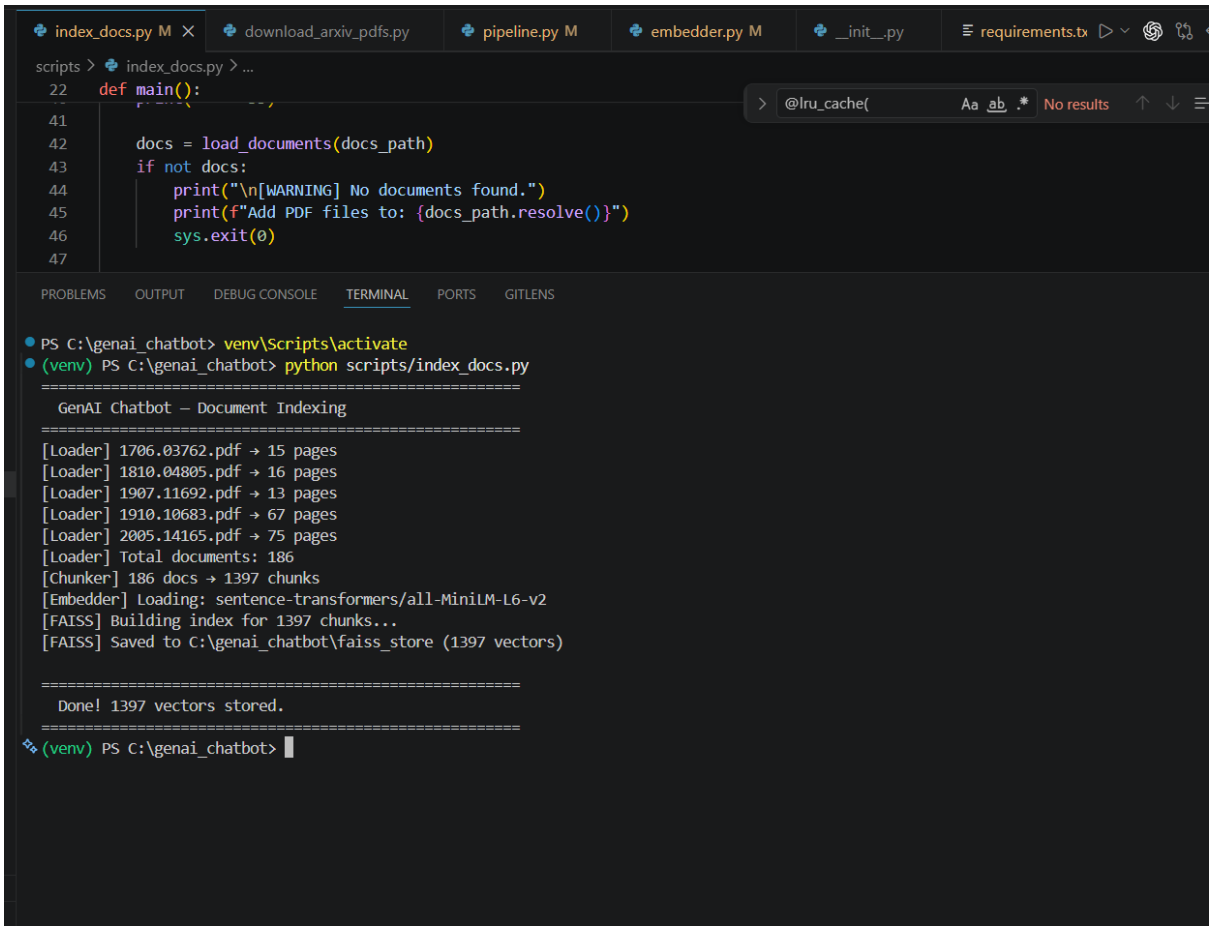
- scripts/index_docs.py

The project also includes:

- scripts/download_arxiv_pdfs.py

This script combines downloading, loading, chunking, and index building for the required arXiv documents.

Chunking , Building Index and storing In FAISS can be verified from below screenshot :-



2.5 Retrieval-Augmented Generation Pipeline

The retrieval and answer-generation logic is implemented in rag/pipeline.py. The main method is query_rag().

Relevant methods:

- get_vectorstore()
- reload_vectorstore()
- _format_docs()
- _retrieve_with_threshold()
- _is_low_signal_answer()
- query_rag(query: str, llm: BaseLanguageModel, k: int = 3, stream_callback=None)

The retrieval pipeline works as follows:

1. Load the FAISS vector store via get_vectorstore().
2. Retrieve candidate chunks using _retrieve_with_threshold().
3. Format those chunks into a prompt-ready context string with _format_docs().
4. Use RAG_PROMPT to instruct the model to answer only from the provided context.
5. Run the chain: prompt | llm | StrOutputParser().
6. Filter out weak, empty, or abstaining outputs using _is_low_signal_answer().

2.6 Open-Source Language Model Integration

The project integrates open-source language models through a router pattern in models/llm_router.py.

Relevant method:

- get_llm() -> tuple[BaseLanguageModel, str]

The LLM selection strategy is:

- primary: Ollama local model
- fallback: Hugging Face CPU model

Ollama Integration

Implemented in models/ollama_model.py.

Relevant methods:

- is_ollama_running()
- is_ollama_model_available(model_name: str)
- get_ollama_llm() -> ChatOllama
- ollama_generate(prompt: str) -> str

Default Ollama model:

- llama3.2:1b

This path supports local inference and avoids paid API dependencies.

Hugging Face Integration

Implemented in models/hf_model.py.

Relevant class and methods:

- class HuggingFaceCPULLM(LLM)
- _load_pipeline()
- _call()
- count_tokens()
- get_hf_llm()

Default HF model:

- TinyLlama/TinyLlama-1.1B-Chat-v1.0

This fallback is useful when Ollama is unavailable. The implementation runs on CPU and is wrapped in a LangChain-compatible custom LLM class.

2.7 Conversational Memory

Conversational memory is implemented
across conversation/chat_handler.py and conversation/session_store.py.

Memory Injection

In conversation/chat_handler.py, the following methods construct and apply recent history:

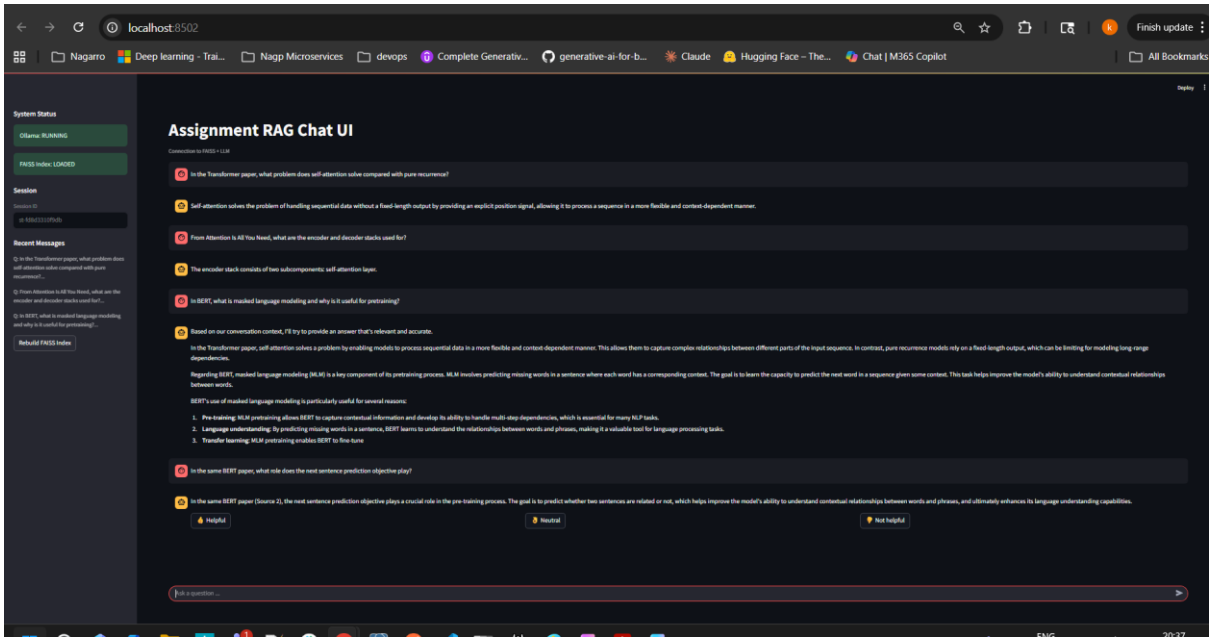
Relevant methods:

- _build_memory_context(session_id: str, turns: int = _MEMORY_TURNS)
- _augment_input_with_memory(user_input: str, session_id: str)
- handle_query(user_input: str, session_id: str = "default", stream_callback=None)

The constant:

- _MEMORY_TURNS = 4

Conversation memory retaining 4 conversation , can be verified from streamlit UI attached screenshot below



The application fetches the most recent four turns and prepends them to the current query when relevant. This allows follow-up questions to be interpreted in conversational context.

Memory Storage and Retention

Implemented in conversation/session_store.py using an in-memory store backed by module-level dictionaries.

Relevant methods:

- **_enforce_history_retention(session_id: str)**
- **log_to_db(query: str, response: dict, session_id: str = "default")**
- **get_history(session_id: str = "default", limit: int = 20)**
- **set_feedback(chat_id: int, score: int, note: str = "")**

The retention policy is enforced by _enforce_history_retention(), which keeps only the latest four turns per session based on settings.history_retention_turns. Each turn is stored with metadata including query, response, LLM source, and optional feedback score. Within a session, turns older than the retention window are automatically pruned on every new query.

2.8 User Interface

The front end is implemented in ui/app.py using Streamlit.

Relevant methods:

- init_backend()

- _trim_ui_history()

The UI:

- initializes the vector store,
- displays system status,
- provides a chat interface,
- streams model output token by token,
- shows recent session history,
- supports feedback buttons,
- mirrors the 4-turn retention policy in session state.

This creates a simple, usable interface for testing the full RAG pipeline end to end.

---

3. Evaluation

3.1 Test Question Set

The project includes a predefined evaluation suite in scripts/eval_questions.json. The file contains 10 questions with the fields:

- id
- question
- expected_keywords
- requires_context

The questions cover major themes from the indexed papers, including:

- In the Transformer paper, what problem does self-attention solve compared with pure recurrence?
- "From Attention Is All You Need, what are the encoder and decoder stacks used for?"
- "In BERT, what is masked language modeling and why is it useful for pretraining?"
- "In the same BERT paper, what role does the next sentence prediction objective play?"
- "Based on your previous answer, summarize how MLM and NSP complement each other in BERT."
- "In the GPT-3 paper, what is in-context learning and how does it differ from fine-tuning?"

- "According to GPT-3, how do zero-shot, one-shot, and few-shot prompting differ?"
- "In RoBERTa, what major training changes were made relative to the original BERT recipe?"
- "How does dynamic masking in RoBERTa differ from static masking in BERT?"
- "In T5, what does the text-to-text framework mean for handling different NLP tasks?"

3.2 Evaluation Script

The main evaluator is:

- scripts/run_integration_eval.py

Relevant methods:

- _load_questions()
- _ask()
- _ask_direct()
- _check_api_health()
- _check_ragas_dependencies()
- _configure_ragas_local_llm()
- _try_ragas()
- main()

3.3 Metrics and Scoring

RAGAS evaluation

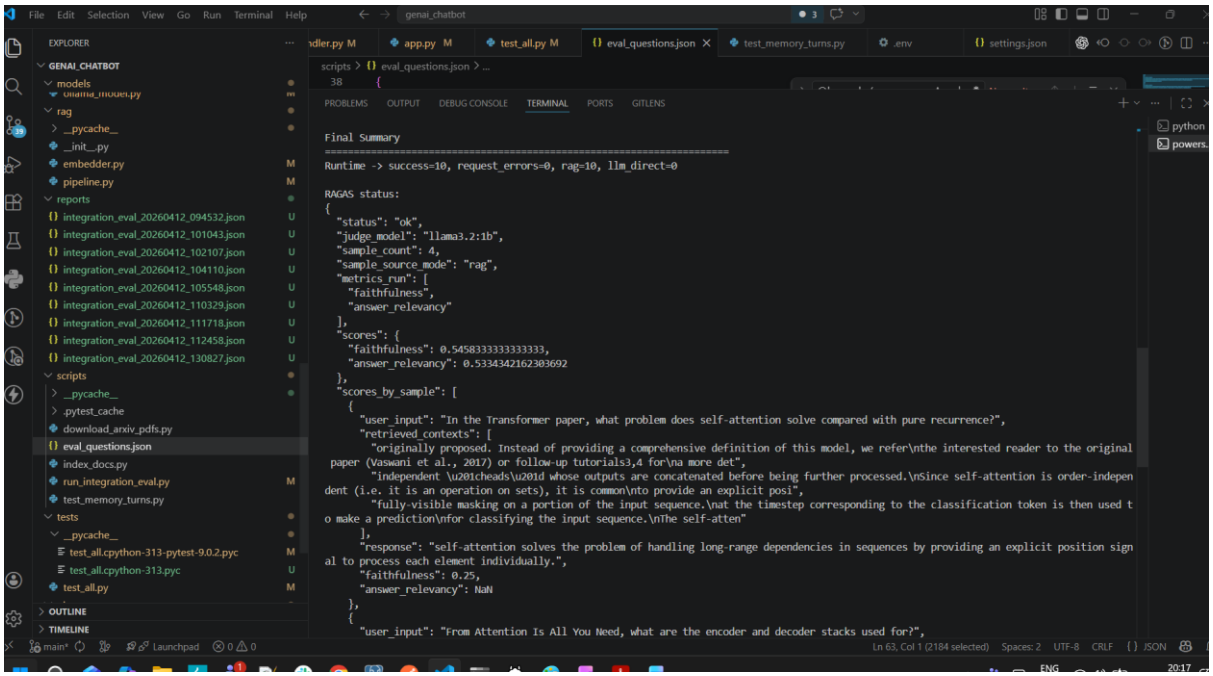
The _try_ragas() method in scripts/run_integration_eval.py attempts to evaluate responses using:

- faithfulness
- answer_relevancy
- contextual_awareness
- response_quality

The actual metric set executed is adaptive: under parser-safe mode with a small local judge model (typically < 2B parameters), the evaluator may execute only the core metrics

(faithfulness and answer_relevancy) and backfill the remaining scores via deterministic heuristics based on answer length, relevance overlap, and context grounding.

Please find attached screenshot for the same :-



3.4 Memory Evaluation

The repository also includes:

- scripts/test_memory_turns.py

This script is designed to test the 4-turn retention behavior by sending more than four messages and checking whether older memory is discarded.

4. Analysis of Performance

4.1 Positive Results

The project successfully demonstrates the core technical requirements of a RAG system:

- The five required PDFs are present and ingested.
- Text extraction and chunking are implemented.
- Embeddings are generated using an open-source model.
- A FAISS vector database is created and stored locally.
- Retrieval and generation are integrated in a working pipeline.
- An open-source LLM is used through Ollama, with Hugging Face fallback.
- Conversational memory over the last four turns is implemented.

- A 10-question evaluation suite exists.
- Evaluation reports have been generated and saved.

The latest report indicates:

- question_count = 10
- successful_responses = 10
- request_errors = 0
- rag_responses = 9
- llm_direct_responses = 1

4.2 Potential Challenges

1. local-judge quality and parser robustness can influence metric stability across runs which may cause Evaluation variance risk
2. when retrieval confidence is low, direct-generation fallback can still produce fluent but weakly grounded answers

4.3 Potential Improvements :-

1. To constrain retrieval using metadata filters (for example, BERT-only when question intent is BERT).
2. Expand observability i.e log retrieval scores, chosen chunks, fallback trigger reasons, and prompt mode per question
3. To add a dedicated memory stress test script and assertions for retention edge cases and cross-turn leakage.

5. Conclusion

This project delivers a working local RAG chatbot that ingests five PDFs, builds a FAISS vector database, retrieves semantically relevant chunks, and answers user questions using an open-source language model. It also implements short-term conversational memory over the last four turns, satisfying the main architectural goals of the assignment.It also involves important implementation elements include the Settings class, the HuggingFaceCPU LLM class, and core methods such as load_documents(), chunk_documents(), build_index(), query_rag(), handle_query(), and _enforce_history_retention().

The evaluation artifacts show that the system is operational and capable of answering the predefined question set. Overall, the project demonstrates a solid end-to-end RAG implementation and provides a strong foundation for further refinement.